INSTITUTO TECNOLÓGICO SUPERIOR TECSUP

DISEÑO Y DESARROLLO DE SOFTWARE

CURSO: DESARROLLO DE SOLUCIONES EN LA NUBE

PROYECTO:

LeGrandHotel – Sistema web de gestión de hospedaje

ENTREGA N.º 1 (25%)

AREQUIPA, 2026

Docente:

VÍCTOR CALVO

Presentado por:

- Huacca Ccaso, Diego Armando
- Marcani Leyva, Joseph Fredy
- Pinto Garate, Leo Mathiu
- Quispe Encarnación, Rusell

Arequipa – Perú

2026

Índice

1. Alcance de la entrega del 25% 3
2. Descripción del proyecto 3
2.1 Tecnologías 3
3. Arquitectura y entorno cloud 3
3.1 Recursos del entorno (Terraform) 4
3.2 Costo estimado 4
3.3 Capturas de la consola de AWS 5
4. Endpoints definidos 5
4.1 Formato único de respuesta 5
4.2 Códigos HTTP 6
5. Registro y login diferenciados 6
5.1 Pruebas automáticas del registro y login 8
6. Registro de incidentes 8
7. Conclusiones de la entrega 9
8. EXTRAS (adicionales a lo evaluado en el 25%) 10
8.1 EXTRA: Diagrama de base de datos 10
8.2 EXTRA: Pruebas de rendimiento y estrés con k6 (varias corridas) 10
8.3 Análisis y cuello de botella encontrado 13
8.4 EXTRA: Cifrado (ofuscación) del código 14
8.5 EXTRA: Vista del programador 14
8.6 EXTRA: Seguridad y calidad adelantadas 14
9. Siguientes pasos (entrega del 80%) 15

1. Alcance de la entrega del 25%

Esta entrega presenta únicamente los puntos evaluados para la semana 9, según el documento «Pasos para presentación de entregables semana 9-11-13». Todo lo demás que ya se avanzó se presenta al final, en la sección 8, marcado como EXTRA, tal como pidió el docente.

Punto evaluado (25%)	Cómo se cumple	Evidencia en este informe	Estado
Entorno cloud preparado (cuenta AWS, VPC, EC2, RDS)	Infraestructura como código con Terraform: VPC con subredes públicas y privadas, EC2, RDS MySQL 8, Security Groups y SSM	Sección 3: diagrama, recursos y capturas de la consola	Código listo; creación en la cuenta AWS pendiente de aprobación de costo
Endpoints definidos	Rutas en api/routes/api.php bajo /api/v1, formato JSON único y especificación OpenAPI	Sección 4	Cumplido
Registro y login diferenciados y funcionales	POST /auth/register y POST /auth/login con validaciones propias y pantallas separadas /registro y /login	Sección 5 (flujo, capturas y pruebas automáticas)	Cumplido

Repositorio: https://github.com/Josephml23/LeGrandHotel

2. Descripción del proyecto

LeGrandHotel es un sistema web de gestión de hospedaje. Cubre habitaciones y tipos, reservas y estadías (check-in y check-out), cochera, clientes, personal e inventario. Lo usan cinco tipos de usuario: Administrador, Recepcionista, Almacenero, Cliente y Programador.

2.1 Tecnologías

Capa	Tecnología
Backend (API)	Laravel 12 · PHP 8.3 (Docker) · Sanctum (tokens) · Socialite (Google) · Pest (pruebas)
Frontend	React 18 · TypeScript · Vite · React Router · TanStack Query · Axios · Vitest
Base de datos y caché	MySQL 8 en Amazon RDS · Redis (ElastiCache, entrega 80%)
Contenedores	Docker + docker-compose (Nginx, PHP-FPM, MySQL, Redis en local)
Nube	AWS: VPC, EC2, RDS, S3, CloudFront, ALB, CloudWatch, SSM, ECR
Pruebas de carga	k6 con reporte HTML y gráficos
Repositorio	Monorepo: api/, web/, infra/, tests/k6/, docs/

3. Arquitectura y entorno cloud

El usuario solo llega a CloudFront (sitio web) y al balanceador (API). La API corre en EC2 y la base de datos no tiene acceso desde internet: vive en subredes privadas y solo acepta conexiones desde las instancias de la API.

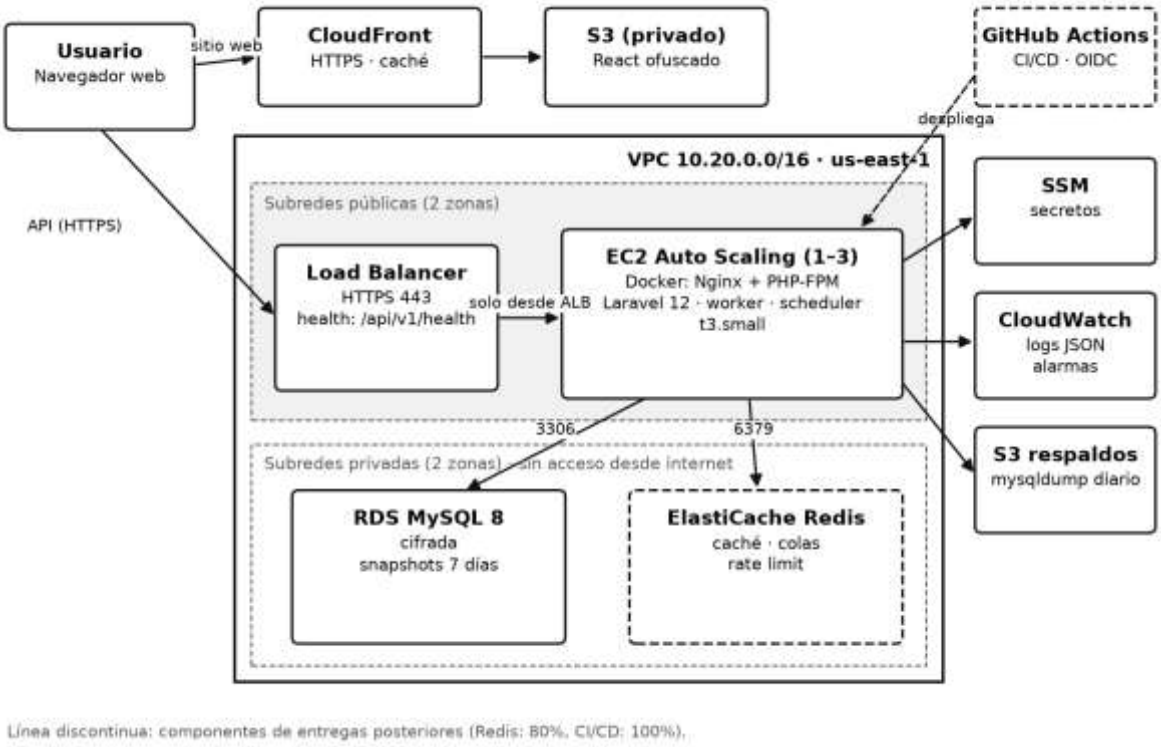


Figura 1. Arquitectura de LeGrandHotel en AWS (us-east-1).

3.1 Recursos del entorno (Terraform)

Todo el entorno está descrito como código en infra/terraform, así se puede crear, revisar y destruir de forma repetible y sin pasos manuales.

Recurso	Configuración	Archivo
VPC	10.20.0.0/16, 2 subredes públicas y 2 privadas en 2 zonas de disponibilidad	network.tf
EC2	t3.small con Docker (Nginx + PHP-FPM), Auto Scaling de 1 a 3 instancias	compute.tf
RDS	MySQL 8, db.t3.micro, 20 GB gp3 cifrado, subred privada, snapshots automáticos de 7 días	data.tf
Security Groups	ALB: 443/80 · EC2: solo desde el ALB · RDS: 3306 solo desde EC2	network.tf
SSM Parameter Store	Secretos y variables de producción (contraseña de BD como SecureString)	data.tf
Presupuesto	Alerta de USD 10 desde el primer día	monitoring.tf

3.2 Costo estimado

Recurso	USD/mes (24/7)
EC2 t3.small	≈ 15
RDS db.t3.micro + 20 GB	≈ 15
IPv4 públicas	≈ 4–8

Recurso	USD/mes (24/7)
Total 25% (sin ALB ni Redis)	≈ 35; ≈ 12 apagando EC2 y RDS fuera de horario

3.3 Capturas de la consola de AWS

[PENDIENTE] *Insertar aquí las capturas de la consola cuando se ejecute «terraform apply»: (1) VPC con sus 4 subredes, (2) instancia EC2 en estado «running», (3) base de datos RDS «Available» en subred privada, (4) Security Groups. El equipo decidirá la creación una vez aprobado el costo; el código ya está listo y validado por el pipeline.*

4. Endpoints definidos

Todas las rutas viven bajo /api/v1 y responden JSON. Salvo autenticación y salud, exigen un token de Sanctum. Los listados aceptan ?page= y ?per_page= (máximo 50). La especificación completa está en docs/endpoints/openapi.yaml y se puede abrir en editor.swagger.io.

Módulo	Método y ruta	Acceso
Salud	GET /health	Público
Autenticación	POST /auth/register	Público (3/min)
Autenticación	POST /auth/login	Público (5/min)
Autenticación	GET /auth/me · POST /auth/logout · POST /auth/refresh	Autenticado
Autenticación	GET /auth/google/redirect · GET /auth/google/callback	Público
Tipos de habitación	GET, POST /room-types · GET, PUT, DELETE /room-types/{id}	Admin (escritura), Recepción (lectura)
Habitaciones	GET, POST /rooms · GET, PUT, DELETE /rooms/{id} · PATCH /rooms/{id}/status	Admin, Recepción
Habitaciones	GET /rooms/available?from=&to=	Admin, Recepción, Cliente
Clientes	CRUD /clients · GET, PUT /me/client	Admin, Recepción · Cliente
Reservas y estadías	CRUD /reservations · POST /reservations/{id}/check-in · POST /stays/{id}/check-out	Admin, Recepción, Cliente (solo las suyas)
Cochera	CRUD /parking-spots · CRUD /parking-assignments	Admin, Recepción
Inventario	CRUD /inventory-items · POST /inventory-items/{id}/movements	Admin, Almacenero
Personal y usuarios	CRUD /staff · CRUD /users · PUT /users/{id}/roles	Admin
Reportes	GET /reports/occupancy · GET /reports/income	Admin
Programador	GET /dev/overview · /dev/routes · /dev/requests · /dev/errors	Programador, Admin

4.1 Formato único de respuesta

```
Éxito: { "success": true, "data": {...}, "meta": { "page": 1, "per_page": 15, "total": 120 } }
```

```
Error: { "success": false, "error": { "code": "ROOM_ALREADY_BOOKED", "message": "...",
"details": {...} } }
```

4.2 Códigos HTTP

Código	Cuándo se devuelve
200 / 201 / 204	Lectura, creación y eliminación correctas
400	Cuerpo JSON mal formado
401	Sin token, token vencido o credenciales incorrectas
403	El rol no tiene permiso (aunque la ruta se escriba a mano)
404	Recurso o ruta inexistente (p. ej. ROOM_NOT_FOUND)
409	Conflicto: habitación ya reservada en esas fechas
422	Validación fallida (detalle por campo)
429	Demasiadas peticiones, con cabecera Retry-After
500 / 503	Error interno o servicio caído: mensaje claro y código de incidente, sin trazas

5. Registro y login diferenciados

El registro y el inicio de sesión son dos endpoints, dos validaciones (FormRequest) y dos pantallas distintas:

	Registro	Login
Endpoint	POST /api/v1/auth/register	POST /api/v1/auth/login
Pantalla	/registro	/login
Datos	Nombre, correo, contraseña (2 veces), tipo y número de documento, teléfono, nacionalidad	Correo y contraseña
Validación	Correo y documento únicos; contraseña ≥ 8 con letras y números	Credenciales contra el hash bcrypt
Resultado	Crea la cuenta con rol Cliente y su ficha de huésped; envía correo de verificación; devuelve token	Devuelve token Bearer de 60 minutos y los roles del usuario
Errores	422 con detalle por campo; 429 al 4.º intento por minuto	401 credenciales incorrectas; 429 al 6.º intento por minuto

La contraseña nunca se guarda en texto plano: el modelo User usa el cast «hashed» (bcrypt, 12 rondas). Tras iniciar sesión, cada rol entra a su propio inicio: el personal al Panel, el Cliente a «Mis reservas» y el Programador a su vista de diagnóstico.



Figura 2. Pantalla de inicio de sesión (/login).

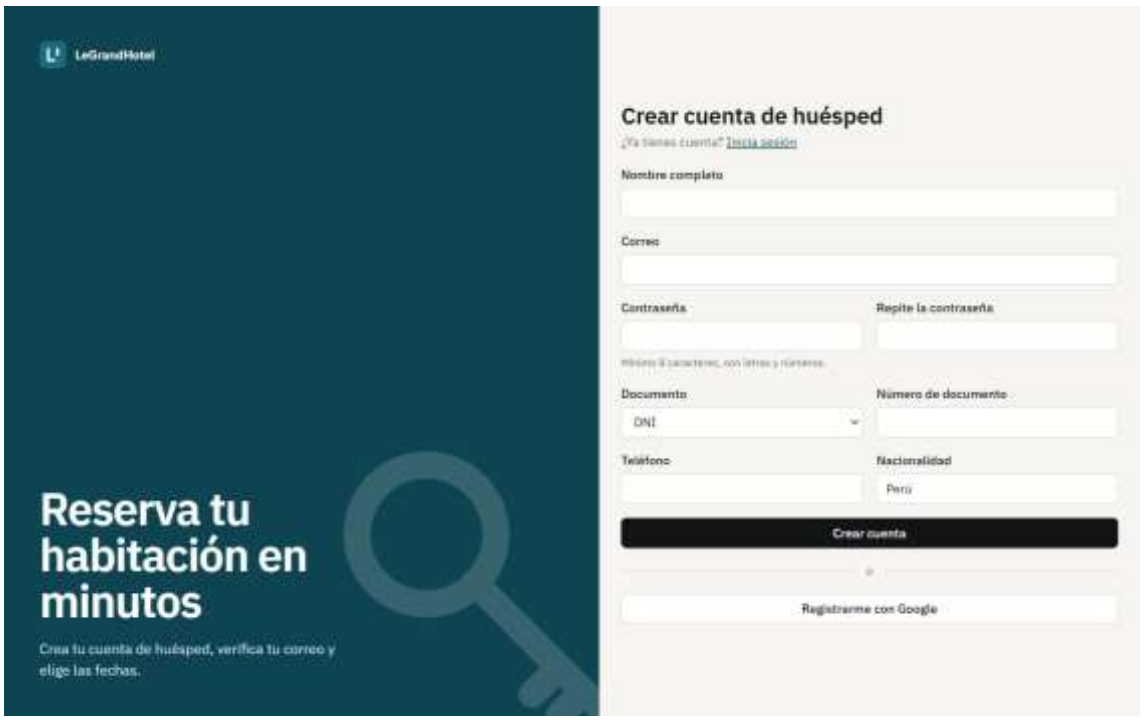


Figura 3. Pantalla de registro de huésped (/registro), separada del login.

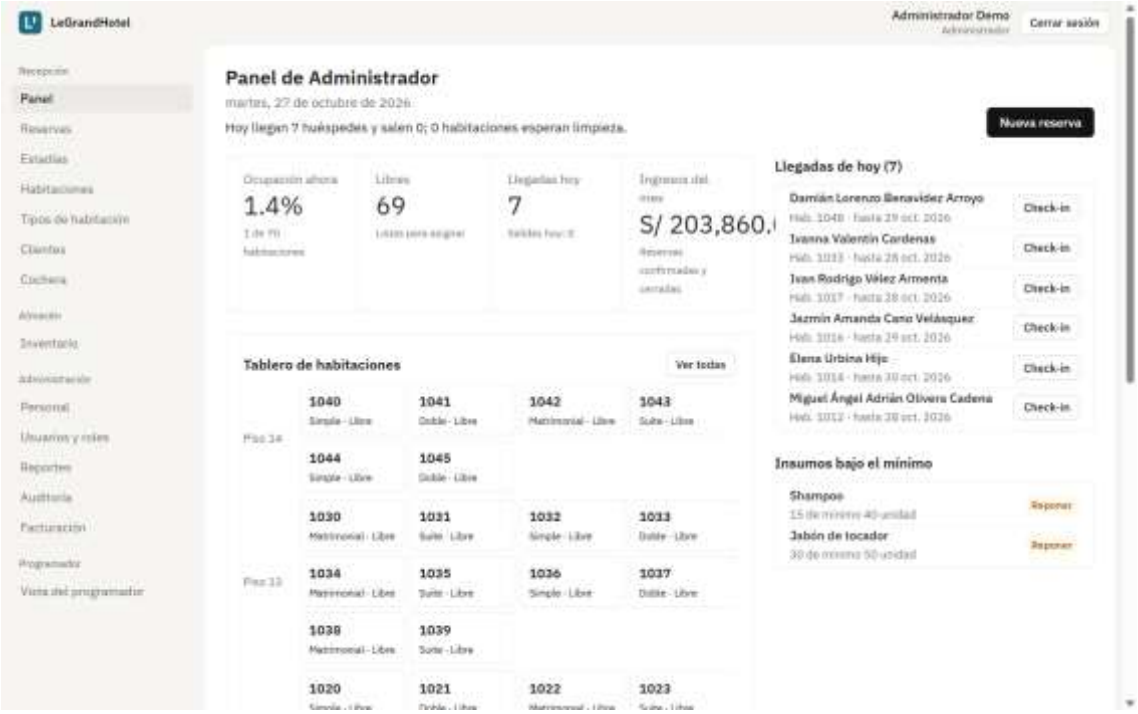


Figura 4. Panel de recepción después de iniciar sesión como Administrador.

5.1 Pruebas automáticas del registro y login

Prueba (api/tests/Feature/AuthTest.php)	Resultado
Registro crea un Cliente con contraseña bcrypt y devuelve token	Pasa
Registro con datos inválidos responde 422	Pasa
No se permite registrar un correo repetido	Pasa
Registro limitado a 3 por minuto (429 con Retry-After)	Pasa
Login con credenciales válidas devuelve token y roles	Pasa
Login con contraseña incorrecta responde 401	Pasa
Login limitado a 5 intentos por minuto	Pasa
Sin token o con token vencido responde 401	Pasa
Frontend: login y registro son pantallas separadas; errores 401 y 422 visibles (Vitest)	Pasa

Resultado global: 89 pruebas de la API (376 aserciones) y 14 pruebas del frontend pasan. El análisis estático con PHPStan no reporta errores y el pipeline de GitHub Actions está en verde.

6. Registro de incidentes

Toda caída, aunque sea en desarrollo, se registra el mismo día (docs/incidentes.md).

Fecha	Qué falló	Motivo	Solución	Tiempo
26/10/2026	El login devolvía 500 con código de incidente XG1R7UASWT	«php artisan serve» lanzaba PHP sin la extensión pdo_sqlite	Servidor local con php -S cargando la extensión	≈ 2 min

Fecha	Qué falló	Motivo	Solución	Tiempo
26/10/2026	El frontend mostró la pantalla 503	El servidor apuntaba a un index.php inexistente	Usar api/public/index.php como router; la pantalla 503 reintentó sola	≈ 1 min
27/10/2026	Primera corrida de k6: 100% de peticiones fallidas	PHP escuchaba solo en IPv6 (::1) y k6 resolvía localhost a IPv4	BASE_URL=http://[::1]:8000; se repitieron las corridas	≈ 5 min

7. Conclusiones de la entrega

- Los tres puntos del 25% están implementados: endpoints definidos y documentados, registro y login separados y probados, y el entorno cloud descrito como código.
- Queda pendiente crear los recursos en la cuenta AWS y agregar las capturas de la consola (sección 3.3), una vez aprobado el costo.
- Las pruebas de carga locales (sección 8.2) muestran que el servidor de desarrollo se satura con pocos usuarios. En la entrega del 80% se repetirán en AWS con Nginx + PHP-FPM, que atiende varias peticiones a la vez.

8. EXTRAS (adicionales a lo evaluado en el 25%)

Todo lo de esta sección es EXTRA: no forma parte de lo evaluado en el 25% y se presenta como avance adicional.

8.1 EXTRA: Diagrama de base de datos

13 tablas del dominio en MySQL 8 con los índices que pide el plan: rooms(status), reservations(check_in, check_out), reservations(client_id), clients(document_number) único e inventory_items(name). La versión editable está en DBML (docs/diagramas/legrandhotel.dbml) y se abre en dbdiagram.io.

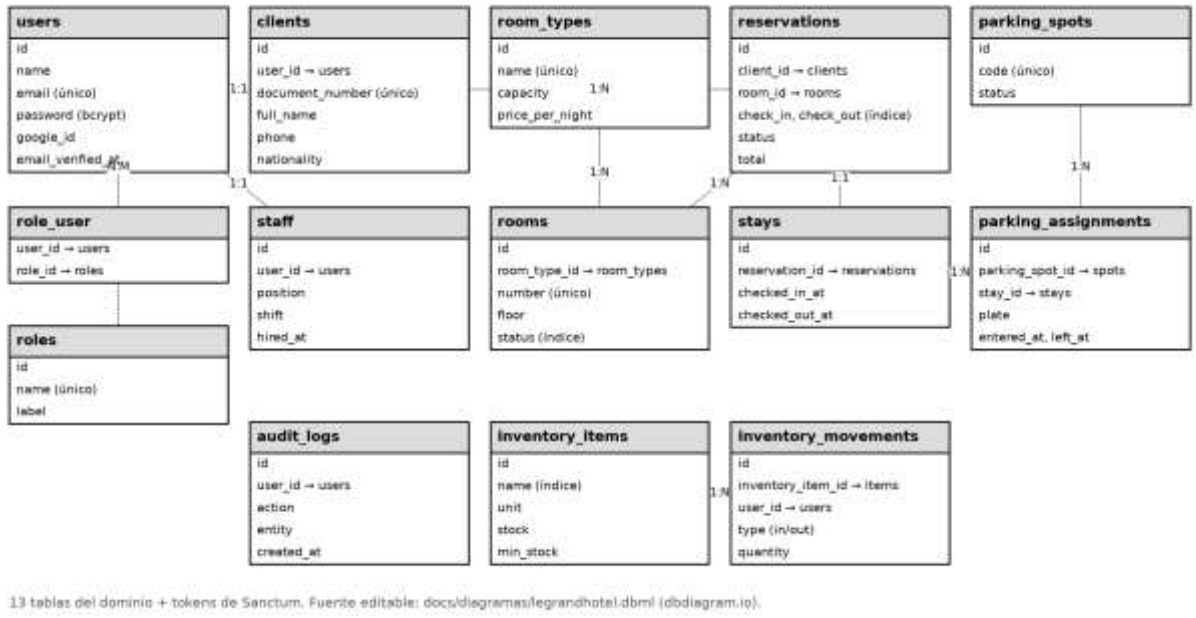


Figura 5. Modelo entidad-relación de LeGrandHotel.

8.2 EXTRA: Pruebas de rendimiento y estrés con k6 (varias corridas)

Se hicieron cinco corridas con k6 contra la API en el equipo de desarrollo (Windows, PHP 8.2, servidor integrado de PHP que atiende una petición a la vez, SQLite y caché en archivo). No es el servidor de producción: sirven para comparar la carga entre corridas y detectar cuellos de botella antes de subir a AWS.

Escenario: 70% lecturas (habitaciones disponibles, mis reservas, tipos de habitación), 20% login y 10% creación de reservas. Cada escalón sube en 10 s, mantiene la carga 45 s y baja en 5 s. Meta del plan: p95 menor a 500 ms y 0% de errores.

Corrida	Peticiones	Errores	p50	p95	Resultado
Smoke (5 VU, 30 s): salud, BD, caché y 401	222	0.00%	203 ms	321 ms	Cumple (100% de verificaciones)

VU	Peticiones	Pet./s	p50	p95	Máx.	Errores	p95 < 500 ms
5	350	4.4	288 ms	528 ms	1,804 ms	0.00%	No cumple

VU	Peticiones	Pet./s	p50	p95	Máx.	Errores	p95 < 500 ms
10	413	5.2	910 ms	1,309 ms	1,598 ms	0.00%	No cumple
25	438	5.2	3,361 ms	3,952 ms	4,197 ms	0.00%	No cumple
50	483	5.3	7,016 ms	7,885 ms	8,195 ms	0.00%	No cumple

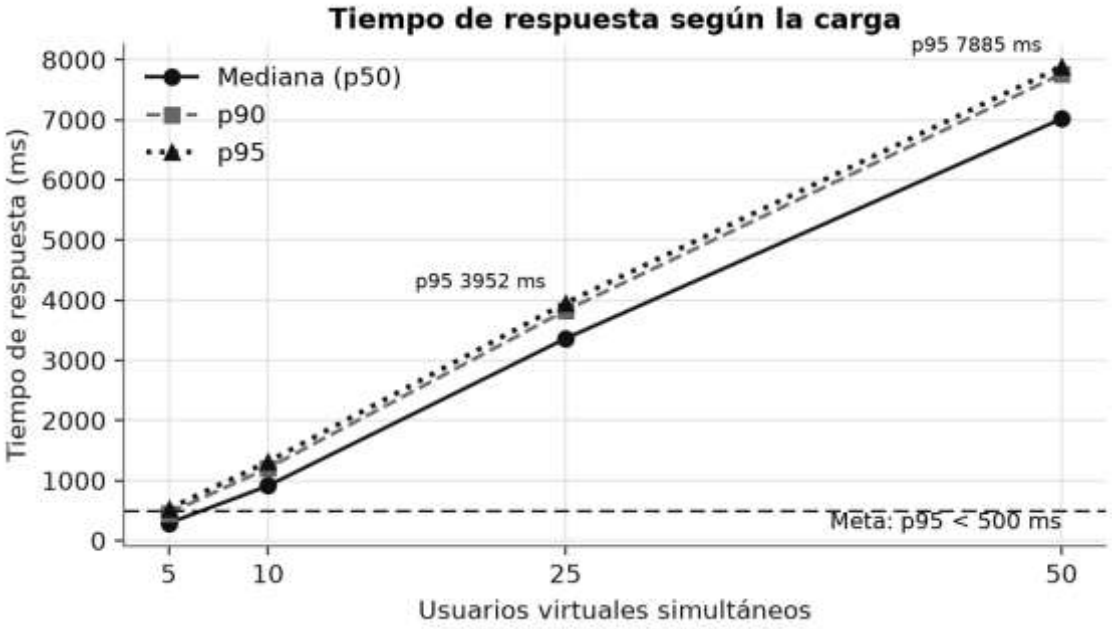


Figura 6. Tiempo de respuesta (p50, p90, p95) según la cantidad de usuarios virtuales.

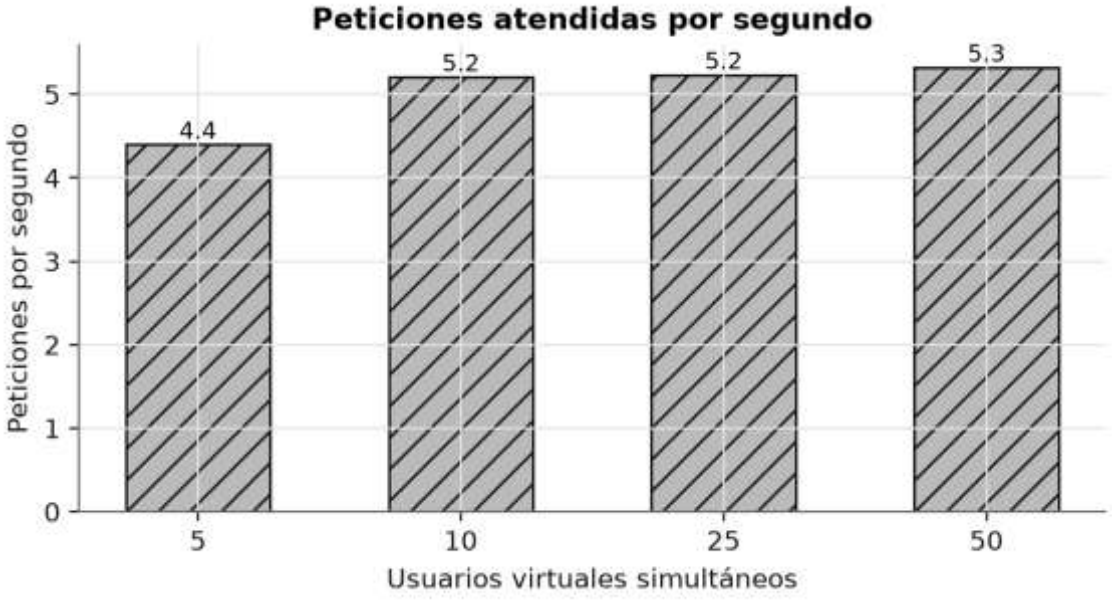


Figura 7. Peticiones atendidas por segundo en cada corrida.



Figura 8. Porcentaje de peticiones fallidas en cada corrida.

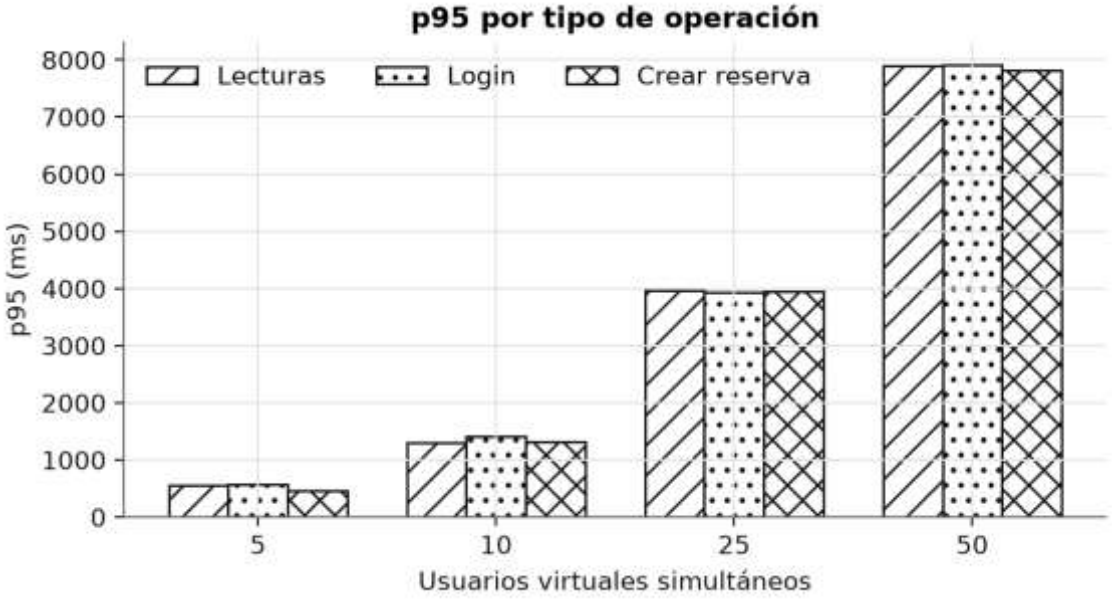


Figura 9. p95 por tipo de operación (lecturas, login, crear reserva).

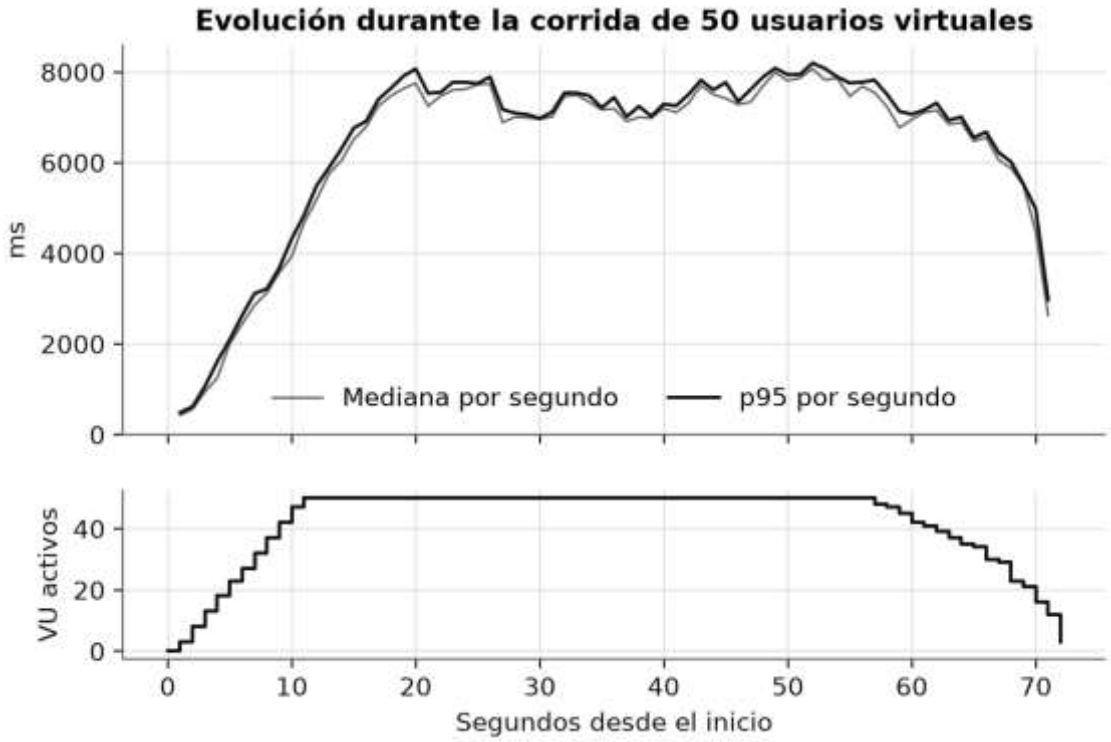


Figura 10. Evolución de la latencia segundo a segundo en la corrida de 50 usuarios.

8.3 Análisis y cuello de botella encontrado

- Con 5 usuarios el p95 fue 528 ms; con 50 usuarios llegó a 7,885 ms. Las peticiones por segundo apenas suben (4.4 → 5.3): el servidor ya estaba al máximo de lo que puede atender.
- Causa: el servidor integrado de PHP en Windows procesa una petición a la vez. Cuando llegan más usuarios, las peticiones esperan en cola y el tiempo de respuesta crece de forma casi lineal, aunque el código de cada endpoint sea rápido (en la prueba masiva de Pest, el listado de 5000 reservas responde en menos de 500 ms).
- Con 50 VU, lecturas, login y creación de reservas tienen casi el mismo p95 (7,890, 7,905 y 7,820 ms). El tiempo lo domina la espera en la cola del servidor, no el trabajo de cada endpoint.
- A pesar de la saturación no hubo errores (0% en todas las corridas): la API responde más lento, pero no se cae ni pierde peticiones.
- Corrección prevista para el 80%: Docker con Nginx + PHP-FPM (20 procesos), OPcache, MySQL en RDS y Redis para caché y rate limit. Las mismas corridas se repetirán en AWS con 50–100 y luego 1000–5000 usuarios, para comparar el antes y el después.

Los reportes HTML interactivos de cada corrida (generados por k6) están en docs/pruebas/local-*.html y los datos crudos en docs/pruebas/datos/.

8.4 EXTRA: Cifrado (ofuscación) del código

El build de producción del frontend sale ofuscado con javascript-obfuscator: los nombres se reemplazan por identificadores hexadecimales y los textos se guardan codificados en base64. Comparación del mismo texto de la interfaz («No tienes acceso»):

Build	Comando	Veces que aparece legible
Normal	npm run build:plain	1
Ofuscado (producción)	npm run build	0

El backend no se publica: corre en un servidor privado sin acceso al código.

8.5 EXTRA: Vista del programador

El rol Programador tiene una pantalla de diagnóstico (/programador) que muestra, sin entrar al servidor: estado de la aplicación, la base de datos y la caché; versión y configuración (vida del token, rate limits, CORS); latencias p50/p95/p99 calculadas del log JSON; rutas más lentas; últimas peticiones; errores buscables por código de incidente, el mismo que ve el usuario en la pantalla 500; y la lista de todas las rutas de la API con su rol de acceso. No muestra contraseñas ni claves, y una prueba automática lo verifica.

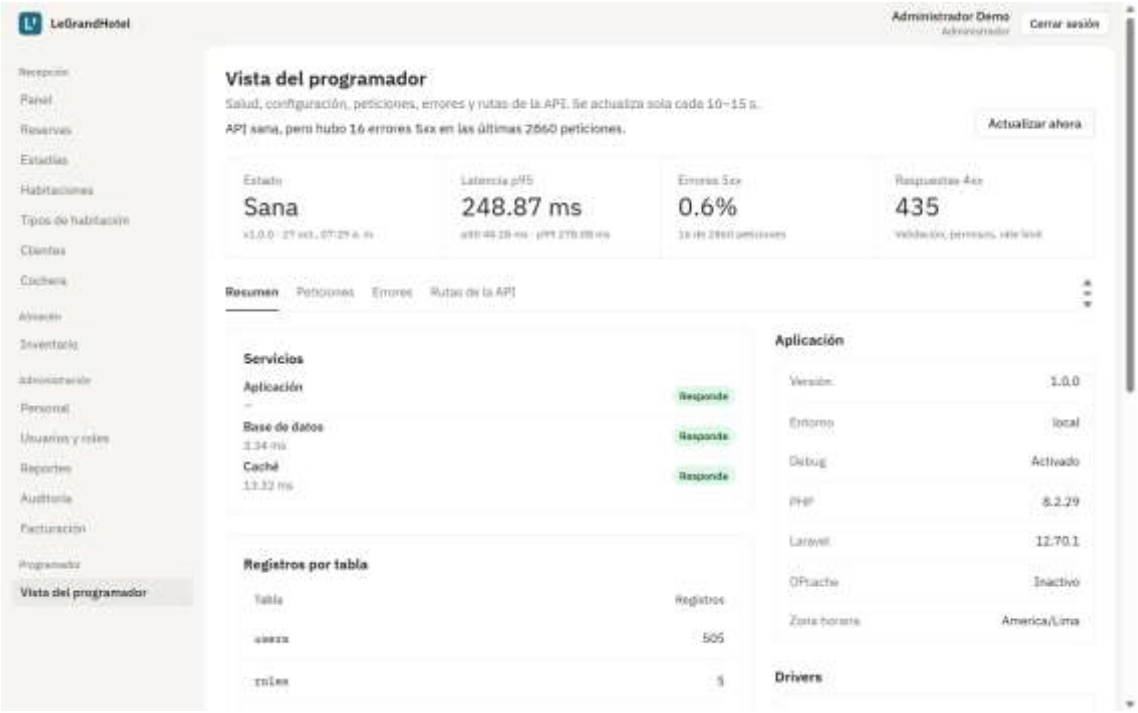


Figura 11. Vista del programador.

8.6 EXTRA: Seguridad y calidad adelantadas

- Rate limits: login 5/min, registro 3/min y API 60/min, con respuesta 429 y Retry-After (punto del 80%).
- Roles y permisos: el rol se valida antes de buscar el registro, así una ruta pedida a mano responde 403 sin revelar si el id existe.

- Pantallas de error en React: 404, «En construcción», 403, sesión expirada, 500 con código de incidente, 503 con reintento cada 30 s, sin conexión y 429 con cuenta regresiva.
- CI/CD en GitHub Actions: estilo (Pint), análisis (PHPStan), pruebas (Pest y Vitest), auditoría de dependencias y Semgrep en cada cambio. Auditoría actual: 0 vulnerabilidades en composer y npm y 0 hallazgos en Semgrep.
- Logs estructurados en JSON (una línea por petición con request_id, ruta, código y duración), listos para CloudWatch.

9. Siguientes pasos (entrega del 80%)

- Crear el entorno en AWS con Terraform (con aprobación del costo) y agregar las capturas.
- Desplegar con el pipeline: imagen Docker en ECR, frontend en S3 + CloudFront, ALB con HTTPS.
- Repetir las pruebas k6 en AWS con 50–100 y 1000–5000 usuarios, con reportes HTML y comparación antes/después.
- Configurar el login con Google (credenciales OAuth) y los respaldos automáticos de RDS y S3.